

LLMOrchestrator

LLMOrchestrator

Eine Steuerungsebene für jeden Headless-CLI-Coding-Agent.

Zusammenfassung

LLMOrchestrator ist ein eigenständiges, wiederverwendbares Go-Modul zum Starten, Verwalten und Kommunizieren mit Headless-CLI-Agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) über ein hybrides Pipe-Datei-Protokoll, ausgestattet mit agentenspezifischen Circuit Breakern, einer pluggbaren Multi-Provider-Auswahl, einer entkoppelten i18n-Abstraktion und Anti-Bluff-Testgarantien.

Kurzbeschreibung

Ein wiederverwendbares Go-Modul, das eine einheitliche Schnittstelle zum Starten und Steuern mehrerer LLM-basierter CLI-Agents über ein hybrides Pipe-Datei-Protokoll bietet. Thread-sicheres Agent-Pooling mit Circuit Breakern und wählbaren Routing-Strategien, bewusst konsumentenunabhängig, mit pluggablem i18n-Übersetzer.

Ausführliche Beschreibung

LLMOrchestrator ist eine gemeinsame Infrastruktur zur Orchestrierung von Headless-CLI-Coding-Agents - die Basiskomponente, die jedes Multi-Agenten-System stillschweigend benötigt und meist schlecht neu implementiert. Statt dass jedes Projekt das Starten von Prozessen, die Nachrichtenformatierung und die Ergebnisauswertung für Tools wie OpenCode, Claude Code, Gemini CLI, Junie und Qwen Code neu erfindet, bietet es eine einheitliche Agent-Schnittstelle, einen thread-sicheren AgentPool und einen MultiProviderPool, der Agents verschiedener Anbieter hinter einer einzigen Fassade bündelt. Das Routing ist

über einen AgentSelector pluggbar – Round-Robin mit Überspringen von Anbietern, die Anforderungen nicht erfüllen, oder priorisierte Auswahl mit Fallback – sodass die Verteilung der Aufgaben eine Frage der gewählten Strategie ist, nicht einer hartcodierten Annahme. Jeder konkrete Agent ist ein schlanker Adapter über einem gemeinsamen BaseAdapter, der den gesamten Prozesslebenszyklus verwaltet: Start mit Pipe-Einrichtung, sanftes Beenden via SIGTERM-gefolgt-von-SIGKILL, Neustart und Liveness – der fehleranfällige Teil, einmal gelöst.

Die Kommunikation ist bewusst hybrid und passt den Transport an die Aufgabe an. Ein Pipe-Transport überträgt zeilengetrennte JSON-Nachrichten mit einer Lese-Timeout pro Anfrage und einer Antwortlängenbegrenzung für schnelle interaktive Nachrichten, während ein Datei-Transport pro Sitzung Inbox/Outbox/Shared-Verzeichnisse für große oder dauerhafte Artefakte nutzt, die nicht in einer Pipe gespeichert werden sollten. Resilienz ist kein nachträglicher Gedanke – sie ist strukturell verankert: Ein Circuit Breaker pro Agent öffnet nach drei aufeinanderfolgenden Fehlern für eine 60-sekündige Abkühlphase, bevor ein Half-Open-Probe durchgeführt wird, und ein Hintergrund-Health-Monitor pingt Agents, sodass ein ausgefallener Agent sich erholen kann, ohne auf eingehenden Traffic warten zu müssen. Die Pool-Anforderung blockiert auf einer Bedingungsvariable, statt die CPU mit Busy-Waiting zu belasten, und der Antwortparser ist zustandslos und sicher für gleichzeitige Aufrufe. Das Modul ist strikt entkoppelt – keine konsumentenspezifischen Details dürfen eindringen – und jede nutzergerichtete Zeichenkette durchläuft einen pluggbaren i18n-Translator, wobei ein NoopTranslator Nachrichten-IDs unverändert zurückgibt, sodass fehlende Übersetzungen deutlich sichtbar werden, statt sich zu verstecken.

Warum wir es entwickelt haben

Jedes Multi-Agenten-System muss CLI-Agents zuverlässig starten und mit ihnen kommunizieren. Die Neuentwicklung von Startprozessen, Nachrichtenformatierung, Parsing und Fehlerbehandlung für jedes Projekt ist ineffizient und fehleranfällig. LLMOrchestrator bündelt diese Funktionen in einem entkoppelten, wiederverwendbaren Modul, dessen spezifische Verantwortung es wiederverwendbar macht – und diese Wiederverwendbarkeit wäre zerstört, sobald konsumentenspezifische Details eindringen.

Warum es ein Game-Changer ist

Es verwandelt das „Betreiben einer Armee heterogener CLI-Agenten“ von einem maßgeschneiderten, projektspezifischen Entwicklungsmarathon in einen einfachen Bibliotheksimport – mit

bereits gelösten und erprobten Lösungen für Pooling, Circuit Breaking, Lebenszyklusmanagement und austauschbare Routing-Mechanismen. Und weil die Anti-Bluff-Tests das reale System Ende-zu-Ende prüfen, statt sich mit „es kompiliert“ zu begnügen, erhält man eine Abstraktion, der man tatsächlich vertrauen kann – selbst unter Nebenläufigkeit und bei Fehlern –, und nicht nur eine, die im Diagramm gut aussieht.

Was innovativ ist

- **Hybrides Pipe- und Dateiprotokoll** – interaktive Geschwindigkeit (JSON-Zeilen über stdin/stdout, Lese-Deadlines, Antwortbegrenzungen) *und* dauerhafter, dateibasierter Austausch (Posteingang/Postausgang/Shared) für große Artefakte, sodass man nie zwischen Latenz und Beständigkeit abwägen muss.
- **Multi-Provider-Pool mit austauschbaren Selektoren** – eine einheitliche Fassade für mehrere CLI-Anbieter, bei der Round-Robin- oder präferenzbasiertes Routing als Richtlinie gewählt wird, statt es fest zu verdrahten.
- **Circuit Breaker pro Agent + Hintergrund-Gesundheitsmonitor** – automatische Degradation *und* Wiederherstellung (3 Fehler → 60 Sekunden offen → halb-offener Test), sodass ein instabiler Agent isoliert und anschließend ohne manuelles Eingreifen wieder eingebunden wird.
- **Pooling ohne aktives Warten** – Acquire blockiert auf sync.Cond, bis ein passender, gesunder Agent verfügbar ist oder der Kontext abgebrochen wird, sodass das Warten keine CPU-Ressourcen verbraucht.
- **Strikte Entkopplung + Anti-Bluff-i18n** – der NoopTranslator gibt Nachrichten-IDs unverändert zurück, sodass fehlende Übersetzungen unmöglich übersehen werden können, statt stillschweigend leer zu bleiben.
- **Sicherheit als Standard** – eine Positivliste für Binärpfade verhindert Shell-Interpolation und damit jede Angriffsfläche für Command-Injection, abgesichert durch Pfad-Traversal-Schutz, eine 1-MiB-Antwortbegrenzung gegen unkontrollierte Ausgaben und API-Schlüsselmaskierung in Logs.
- **Anti-Bluff-Challenge-Harness** – echte Festplatten-/JSON-/Parser-Roundtrips in fünf Sprachen, kombiniert mit einem Mutations-Gate, das bei defekten Features zwingend einen Fehlercode zurückgeben muss – ein Test, der beweist, dass das System tatsächlich scheitern *kann*.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Zuverlässige Agentenprozess-E/A.** Die Kommunikation mit einem gestarteten CLI-Prozess ist trügerisch schwierig; gelöst durch ein hybrides Pipe- und Dateitransportprotokoll, einen definierten Nachrichten-/Parser-Vertrag, damit beide Seiten das Übertragungsformat einheitlich interpretieren, und einen BaseAdapter, der den gesamten Prozesslebenszyklus zentralisiert – inklusive eines eleganten SIGTERM-Timeouts mit SIGKILL-Fallback.
- **Nebenläufigkeit ohne aktives Warten.** Gelöst durch einen AgentPool mit Mutex und Bedingungsvariablen, bei dem Acquire schläft, bis ein passender Agent mit der benötigten Funktionalität tatsächlich frei wird, kombiniert mit einem zustandslosen, nebenwirkungsfreien Parser, der sicher aus mehreren Goroutinen gleichzeitig aufgerufen werden kann.
- **Isolation von Provider-Ausfällen.** Gelöst, sodass ein fehlerhafter Anbieter die anderen nicht in Mitleidenschaft zieht: Circuit Breaker pro Agent begrenzen den Schadensradius, und ein Gesundheitsmonitor-Goroutine treibt die Wiederherstellung voran – selbst wenn keine Anfragen eingehen, die sie auslösen würden.
- **Korrektheit beweisen, nicht nur Kompilierbarkeit.** Gelöst durch einen Challenge-Runner: Dutzende Invarianten in en/sr/ja/es/de, die das reale System testen, plus ein gekoppeltes Mutations-Gate (LLMORCH_MUTATE_RUNNER=1 muss fehlschlagen → Wrapper-Exitcode 99), das die Funktion gezielt beschädigt, um zu beweisen, dass das Gate selbst kein Bluff ist.
- **Lokalisierung ohne stille Fehler.** Gelöst durch die NoopTranslator-Nahtstelle mit unveränderten Nachrichten-IDs und injizierbaren Übersetzern pro Verbraucher, sodass Lücken in den Übersetzungen immer sichtbar sind – statt kaschiert zu werden.

Technologie-Stack

- **Go (1.25)** — ausgewählt wegen seiner erstklassigen Nebenläufigkeit und sauberen Prozesssteuerung, genau das, was die Orchestrierung von Live-Agent-Prozessen erfordert; es implementiert das Modul, seine Agenten-Adapter, Transportschichten und den Parser.
- **Go Standardbibliothek (ergänzt durch testify, yaml.v3)** — eine bewusste Entscheidung, um die Abhängigkeitsoberfläche minimal zu halten und *keine* LLM-SDKs einzubinden, damit das Modul leichtgewichtig bleibt und sich problemlos in jeden Verbraucher einbetten lässt, ohne Anbieter-spezifischen Ballast mitzuschleppen.

- **Pipe-Transport (JSON-lines über stdio)** — gewählt für schnelle interaktive Nachrichtenübermittlung, abgesichert durch Lese-Timeouts und Antwortlängenbegrenzungen, sodass ein blockierter oder außer Kontrolle geratener Agent den Aufrufer nicht lahmlegen kann.
- **Datei-Transport (Inbox/Outbox/geteilt)** — gewählt für den dauerhaften Austausch großer Artefakte pro Sitzung, wo eine Pipe das falsche Werkzeug wäre.
- **sync.Mutex/sync.Cond** — eingesetzt, um eine blockierende, faire Agentenpool-Belegung ohne aktives Warten zu realisieren.
- **Circuit Breaker + HealthMonitor** — kombiniert gewählt, um pro Agenten Resilienz *und* aktive Wiederherstellung zu gewährleisten, nicht nur Fehlererkennung.
- **pkg/i18n Translator** — als entkoppelte Lokalisierungsschnittstelle ausgewählt, die verbraucherspezifische Zeichenketten aus dem Kern heraushält.
- **Challenge-Harness (challenges/runner) + Makefile (test -race, fuzz, cover)** — gewählt für eine faktengestützte, betrugssichere Überprüfung, einschließlich Race-Condition-Erkennung und Parser-Fuzzing, sodass Korrektheit unter adversativen Bedingungen nachgewiesen und nicht einfach vorausgesetzt wird.

Status & Transparenzhinweise

- **Status: Beta.** Ein entkoppeltes, wiederverwendbares Modul, das als Submodul von mehreren Helix/vasic-Projekten genutzt wird. **Lizenz: Apache-2.0**; das GitHub-Repository ist öffentlich.
- Modellmetadaten stammen aus LLMsVerifier, über HelixQA angebunden; dieses Modul importiert LLMsVerifier/VisionEngine/DocProcessor nicht direkt. Die in der `CLAUDE.md` der übergeordneten Anwendung referenzierten Stacks (Gin/PostgreSQL usw.) beschreiben `helix_code`, nicht dieses Modul.

Prioritätsstufe: Helix-primär (LLM-Infrastruktur-Cluster — entkoppeltes, wiederverwendbares Modul). Rangiert nach HelixTrack.